

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS



“P05 Productor Consumidor”

Asignatura:

Sistemas Operativos

Docente:

Ing. Hugo Manuel Barraza Vizcarra

Estudiante:

Grecia Irene Meneses Chaupi

2021-119055

Tacna - Perú

2026

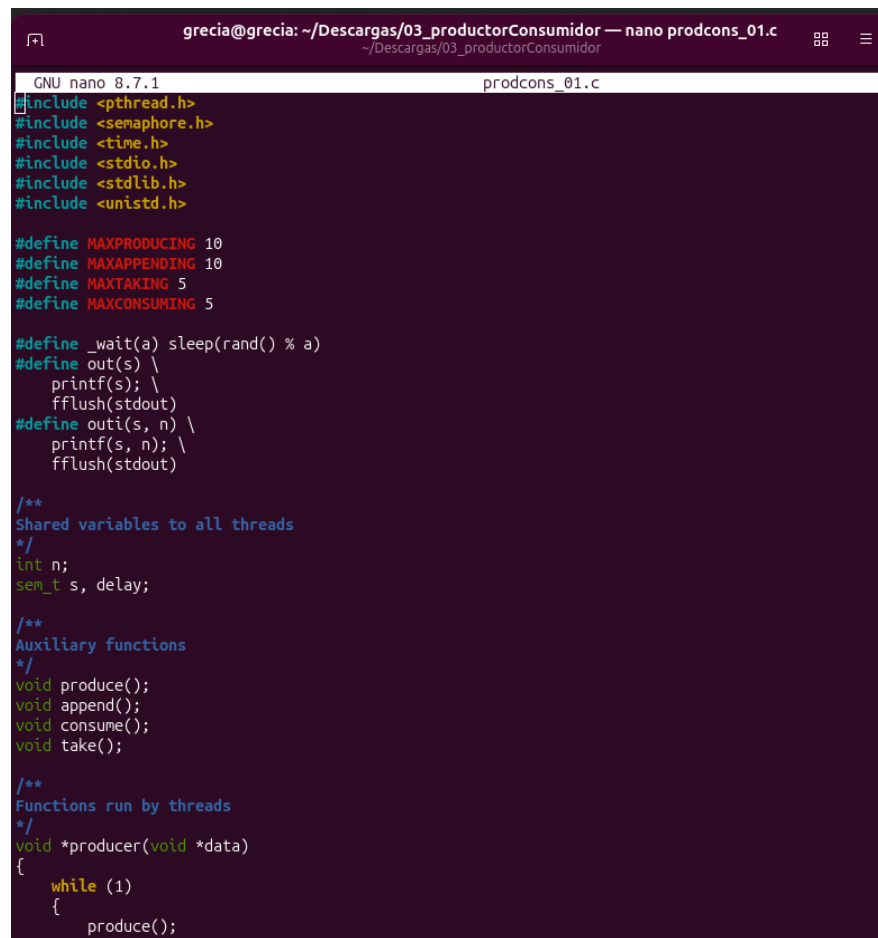
1. Análisis de resultados

Ejecute y analice cada uno de los programas proporcionados, identificando los mecanismos de sincronización empleados y los posibles problemas derivados de su implementación.

Para cada caso, pegue una captura de pantalla y explique el comportamiento observado.

1.1. prodcons_01.c (condición de carrera): Este programa implementa la solución incorrecta del problema productor-consumidor presentada en la figura 5.9 de Stallings. Explique qué problema de sincronización presenta esta implementación.

- **Imagen del código**



```
greCIA@grecia: ~/Descargas/03_productoConsumidor — nano prodcons_01.c
~/Descargas/03_productoConsumidor

GNU nano 8.7.1 prodcons_01.c
#include <pthread.h>
#include <semaphore.h>
#include <time.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

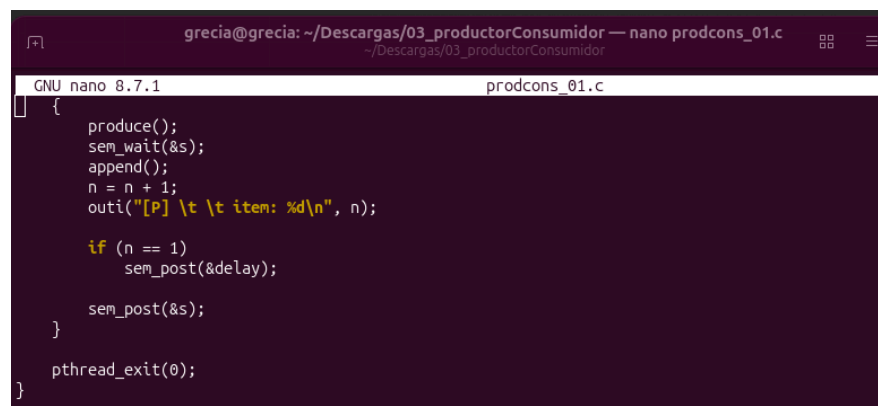
#define MAXPRODUCING 10
#define MAXAPPENDING 10
#define MAXTAKING 5
#define MAXCONSUMING 5

#define _wait(a) sleep(rand() % a)
#define out(s) \
    printf(s); \
    fflush(stdout)
#define outi(s, n) \
    printf(s, n); \
    fflush(stdout)

/**
 * Shared variables to all threads
 */
int n;
sem_t s, delay;

/**
 * Auxiliary functions
 */
void produce();
void append();
void consume();
void take();

/**
 * Functions run by threads
 */
void *producer(void *data)
{
    while (1)
    {
        produce();
```



```
        produce();
        sem_wait(&s);
        append();
        n = n + 1;
        outi("[P] \t \t item: %d\n", n);

        if (n == 1)
            sem_post(&delay);

        sem_post(&s);
    }

    pthread_exit(0);
}
```

```

void *consumer(void *data)
{
    sem_wait(&delay);

    while (1)
    {
        sem_wait(&s);
        take();
        outi("[C] \t \t item: %d\n", n);
        n = n - 1;
        sem_post(&s);
        consume();

        if (n == 0)
            sem_wait(&delay);
    }

    pthread_exit(0);
}

/**
main() function
*/
int main(int argc, char **argv)
{
    pthread_t *consumer_pt, *producer_pt;

```

```

greCIA@grecia: ~/Descargas/03_productorConsumidor — nano prodcons_01.c
~/Descargas/03_productorConsumidor

GNU nano 8.7.1 prodcons_01.c
srand(time(NULL));

sem_init(&s, 0, 1);
sem_init(&delay, 0, 0);

consumer_pt = (pthread_t *)malloc(sizeof(pthread_t));
producer_pt = (pthread_t *)malloc(sizeof(pthread_t));

pthread_create(consumer_pt, NULL, consumer, NULL);
pthread_create(producer_pt, NULL, producer, NULL);

pthread_exit(0);
}

void produce()
{
    out("[P] Producing\n");
    _wait(MAXPRODUCING);
    out("[P] Produced\n");
}

void append()
{
    out("[P] \t Appending\n");
    _wait(MAXAPPENDING);
    out("[P] \t Appended\n");
}

void take()
{
    out("[C] Taking\n");
    _wait(MAXTAKING);
    out("[C] Taken\n");
}

void consume()
{
    out("[C] \t Consuming\n");
    _wait(MAXCONSUMING);
    out("[C] \t Consumed\n");
}

```

- Imagen de la ejecución

```

greCIA@greCIA: ~/Descargas/03_producToRConsumidor
~/Descargas/03_producToRConsumidor
greCIA@greCIA:~/Descargas/03_producToRConsumidor$ nano prodcons_01.c
greCIA@greCIA:~/Descargas/03_producToRConsumidor$ nano prodcons_01.c
greCIA@greCIA:~/Descargas/03_producToRConsumidor$ gcc -o prodcons_01 prodcons_01.c -lpthread
greCIA@greCIA:~/Descargas/03_producToRConsumidor$ ./prodcons_01
[P] Producing
[P] Produced
[P]   Appending
[P]   Appended
[P]     item: 1
[P] Producing
[C] Taking
[C] Taken
[C]     item: 1
[C]   Consuming
[C]   Consumed
[P] Produced
[P]   Appending
[P]   Appended
[P]     item: 1
[P] Producing
[C] Taking
[P] Produced
[C] Taken
[C]     item: 1
[C]   Consuming
[P]   Appending
[P]   Appended
[P]     item: 1
[P] Producing
[C] Consumed
[C] Taking
[P] Produced
[C] Taken
[C]     item: 1
[C]   Consuming
[P]   Appending
[C] Consumed
[P]   Appended
[P]     item: 1
[P] Producing
[C] Taking
[C] Taken
[C]     item: 1
[C]   Consuming
[C] Consumed
[C] Taking
[P] Produced

```

```

greCIA@greCIA: ~/Descargas/03_producToRConsumidor
~/Descargas/03_producToRConsumidor
[C] Consumed
[C] Taking
[P] Produced
[C] Taken
[C]     item: 0
[C]   Consuming
[P]   Appending
[C] Consumed
[P]   Appended
[P]     item: 0
[P] Producing
[C] Taking
[C] Taken
[C]     item: 0
[C]   Consuming
[P] Produced
[P]   Appending
[P]   Appended
[P]     item: 0
[P] Producing
[C] Consumed
[P] Produced
[P]   Appending
[P]   Appended
[P]     item: 1
[P] Producing
[C] Taking
[C] Taken
[C]     item: 1
[C]   Consuming
[C] Consumed
[P] Produced
[P]   Appending
[P]   Appended
[P]     item: 1
[P] Producing
[C] Taking
[P] Produced
[C] Taken
[C]     item: 1
[C]   Consuming
[P]   Appending
[C] Consumed
[P]   Appended
[P]     item: 1

```

```
greCIA@grecia: ~/Descargas/03_producToRConsumidor
~/Descargas/03_producToRConsumidor

[C] Consumed
[C] Taking
[P] Produced
[C] Taken
[C] item: 0
[C] Consuming
[P] Appending
[C] Consumed
[P] Appended
[P] item: 0
[P] Producing
[C] Taking
[C] Taken
[C] item: 0
[C] Consuming
[P] Produced
[P] Appending
[P] Appended
[P] item: 0
[P] Producing
[C] Consumed
[P] Produced
[P] Appending
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[C] Taken
[C] item: 1
[C] Consuming
[C] Consumed
[P] Produced
[P] Appending
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[P] Produced
[C] Taken
[C] item: 1
[C] Consuming
[P] Appending
[C] Consumed
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[C] Taking
```

```
greCIA@grecia: ~/Descargas/03_producToRConsumidor
~/Descargas/03_producToRConsumidor

[P] Produced
[P] Appending
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[C] Taken
[C] item: 1
[C] Consuming
[C] Consumed
[P] Produced
[P] Appending
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[C] Taken
[C] item: 1
[C] Consuming
[C] Consumed
[P] Produced
[P] Appending
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[C] Taken
[C] item: 1
[C] Consuming
[P] Produced
[P] Appending
[C] Consumed
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[C] Taken
[C] item: 1
[C] Consuming
[P] Produced
[P] Appending
[P] Appended
[P] item: 1
[P] Producing
```

```
greceia@grecia: ~/Descargas/03_producutorConsumidor
~/Descargas/03_producutorConsumidor
[C] Taken
[C] item: 1
[C] Consuming
[C] Consumed
[C] Taking
[P] Produced
[C] Taken
[C] item: 0
[C] Consuming
[C] Consumed
[C] Taking
[C] Taken
[C] item: -1
[C] Consuming
[P] Appending
[C] Consumed
[P] Appended
[P] item: -1
[P] Producing
[C] Taking
[C] Taken
[C] item: -1
[C] Consuming
[P] Produced
[P] Appending
[C] Consumed
[P] Appended
[P] item: -1
[P] Producing
[C] Taking
[C] Taken
[C] item: -1
[C] Consuming
[C] Consumed
[C] Taking
[C] Taken
[C] item: -2
[C] Consuming
[P] Produced
[P] Appending
[C] Consumed
[P] Appended
[P] item: -2
[P] Producing
[C] Taking
[P] Produced
[C] Taken
```

- **Explicación del comportamiento**

En la ejecución de prodcons_01.c se observa que al inicio el productor y el consumidor trabajan de manera alternada, teniendo valores cercanos a 0 y 1. Sin embargo, mientras va pasando la ejecución, el consumidor continúa disminuyendo el valor de n incluso cuando este ya llegó a 0, por lo que aparecen valores negativos como -1, -2 y -3. Esto sucede porque el valor de n puede cambiar antes de que el consumidor compruebe si debe detenerse y esperar. Por ello, el productor y el consumidor no mantienen una sincronización correcta y el consumidor puede seguir ejecutándose aunque el valor ya haya llegado a cero.

- 1.2. **prodcons_01_buggy_01.c (error en la operación de un semáforo):** Esta implementación introduce intencionalmente un error al sustituir una operación de espera por una operación de señalización sobre el semáforo delay. Identifique la instrucción incorrecta y explique cómo esta modificación afecta la sincronización entre el productor y el consumidor. ¿Qué sucede si el consumidor intenta retirar un elemento antes de que el productor haya incorporado el primero?

- **Imagen de la ejecución**

```
greCIA@grecia: ~/Descargas/03_productorConsumidor
~/Descargas/03_productorConsumidor

greCIA@grecia:~/Descargas/03_productorConsumidor$ ./prodcons_01_buggy_01
[C] Taking
[P] Producing
[P] Produced
[C] Taken
[C] item: 0
[P] Appending
[C] Consuming
[C] Consumed
[P] Appended
[P] item: 0
[P] Producing
[C] Taking
[P] Produced
[C] Taken
[C] item: 0
[C] Consuming
[P] Appending
[C] Consumed
[P] Appended
[P] item: 0
[P] Producing
[C] Taking
[P] Produced
[C] Taken
[C] item: 0
[C] Consuming
[P] Appending
[C] Consumed
[P] Appended
[P] item: 0
[P] Producing
[C] Taking
[P] Produced
[C] Taken
[C] item: 0
[C] Consuming
[P] Appending
[C] Consumed
[P] Appended
[P] item: 0
[P] Producing
[C] Taking
[C] Taken
[C] item: 0
[C] Consuming
```

```
greCIA@grecia: ~/Descargas/03_productorConsumidor
~/Descargas/03_productorConsumidor

[P] Produced
[P] Appending
[C] Consumed
[P] Appended
[P] item: 0
[P] Producing
[C] Taking
[C] Taken
[C] item: 0
[C] Consuming
[P] Produced
[P] Appending
[C] Consumed
[P] Appended
[P] item: 0
[P] Producing
[C] Taking
[P] Produced
[C] Taken
[C] item: 0
[C] Consuming
[P] Appending
[P] Appended
[P] item: 0
[P] Producing
[C] Consumed
[C] Taking
[P] Produced
[C] Taken
[C] item: 0
[C] Consuming
[P] Appending
[P] Appended
[P] item: 0
[P] Producing
[P] Produced
[P] Appending
[C] Consumed
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[P] Produced
[C] Taken
[C] item: 1
```

```

greCIA@grecia: ~/Descargas/03_productorConsumidor
~/Descargas/03_productorConsumidor

[P]      Appending
[C]      Consumed
[P]      Appended
[P]      item: 1
[P]      Producing
[C]      Taking
[P]      Produced
[C]      Taken
[C]      item: 1
[C]      Consuming
[P]      Appending
[C]      Consumed
[P]      Appended
[P]      item: 1
[P]      Producing
[C]      Taking
[C]      Taken
[C]      item: 1
[C]      Consuming
[C]      Consumed
[P]      Produced
[P]      Appending
[P]      Appended
[P]      item: 1
[P]      Producing
[C]      Taking
[P]      Produced
[C]      Taken
[C]      item: 1
[C]      Consuming
[P]      Appending
[C]      Consumed
[P]      Appended
[P]      item: 1
[P]      Producing
[C]      Taking
[P]      Produced
[C]      Taken
[C]      item: 1
[C]      Consuming
[P]      Appending
[C]      Consumed
[P]      Appended
[P]      item: 1
[P]      Producing
[C]      Taking

```

```

[greceia@grecia: ~/Descargas/03_producutorConsumidor
~/Descargas/03_producutorConsumidor
[C] Taken
[C] item: 1
[C] Consuming
[P] Produced
[P] Appending
[C] Consumed
[P] Appended
[P] item: 1
[C] Taking
[P] Producing
[C] Taken
[C] item: 1
[C] Consuming
[P] Produced
[P] Appending
[C] Consumed
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[C] Taken
[C] item: 1
[C] Consuming
[C] Consumed
[P] Produced
[P] Appending
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[P] Produced
[C] Taken
[C] item: 1
[C] Consuming
[P] Appending
[C] Consumed
[P] Appended
[P] item: 1
[P] Producing
[C] Taking
[C] Taken
[C] item: 1
[C] Consuming
[P] Produced
[P] Appending

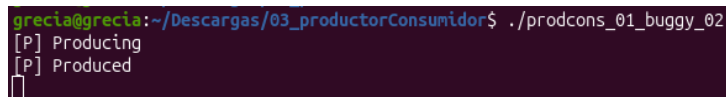
```

- **Explicación del comportamiento**

En la ejecución de `prodcons\_01\_buggy\_01.c` se observa que el consumidor comienza a ejecutarse desde el inicio, incluso antes de que el productor haya agregado correctamente un elemento. Esto ocurre porque en el consumidor se utiliza `sem\_post(&delay)` en lugar de `sem\_wait(&delay)`, por lo que no se realiza la espera inicial que debería controlar su ejecución. Debido a ello, el consumidor puede seguir disminuyendo el valor de `n` aunque este ya haya llegado a 0, generando valores negativos como -1, -2 y -3. En consecuencia, se pierde la sincronización entre el productor y el consumidor.

- 1.3. **prodcons_01_buggy_02.c (interbloqueo por adquisición incorrecta de semáforos):** Esta implementación modifica el orden y la ubicación de las operaciones de sincronización del consumidor. Analice cómo estas modificaciones pueden ocasionar un interbloqueo (deadlock) entre ambos hilos.

- **Imagen de la ejecución**



```
greclia@grecia:~/Descargas/03_productorConsumidor$ ./prodcons_01_buggy_02
[P] Producing
[P] Produced
█
```

- **Explicación del comportamiento**

En esta ejecución solo se muestran las líneas Producing y Produced, y luego el programa deja de avanzar. Esto ocurre porque el consumidor adquiere primero el semáforo s y después queda esperando el semáforo delay. Al mismo tiempo, el productor necesita acceder a s para continuar y poder liberar delay. Como ambos hilos quedan esperando un recurso que el otro necesita, se produce un interbloqueo o deadlock, por lo que la ejecución queda detenida.

- 1.4. **prodcons_01_OK.c:** Este programa implementa la solución presentada en la figura 5.10 de Stallings, que incorpora una variable auxiliar local denominada m. Compare esta implementación con prodcons_01.c y explique cómo la incorporación de dicha variable permite corregir el problema identificado.

- **Imagen de la ejecución**

```
greCIA@greCIA: ~/Descargas/03_productorConsumidor
greCIA@greCIA:~/Descargas/03_productorConsumidor$ gcc -o prodcons_01_OK prodcons_01_OK.c -lpthread
greCIA@greCIA:~/Descargas/03_productorConsumidor$ ./prodcons_01_OK
[P] Producing
[P] Produced
[P]   Appending
[P]   Appended
[P]   item: 1
[C] Taking
[P] Producing
[C] Taken
[C]   item: 1
[C]   Consuming
[C]   Consumed
[P] Produced
[P]   Appending
[P]   Appended
[P]   item: 1
[P] Producing
[C] Taking
[P] Produced
[C] Taken
[C]   item: 1
[C]   Consuming
[P]   Appending
[C]   Consumed
[P]   Appended
[P]   item: 1
[P] Producing
[C] Taking
[P] Produced
[C] Taken
[C]   item: 1
[C]   Consuming
[P]   Appending
[P]   Appended
[P]   item: 1
[P] Producing
[C] Consumed
[C] Taking
[C] Taken
[C]   item: 1
[C]   Consuming
[C]   Consumed
[P] Produced
[P]   Appending
[P]   Appended
```

- **Explicación del comportamiento**

En la ejecución de prodcons_01_OK.c se observa que el productor y el consumidor trabajan de manera coordinada, manteniendo el valor de n en valores válidos como 1, sin llegar a valores negativos. A diferencia del primer programa, esta versión utiliza una variable auxiliar m para guardar el valor de n antes de liberar el semáforo s. De esta manera, el consumidor puede verificar correctamente si debe esperar cuando ya no quedan elementos disponibles. Esto permite evitar el problema de sincronización observado en prodcons_01.c y mantiene una ejecución más estable entre ambos hilos.

1.5. Resuelva el ejercicio 3 (indicado en este enlace) y use como referencia la figura 5.13 para hacer su solución.

- **Imagen de la ejecución**

```

greCIA@greCIA:~/Descargas/03_productorConsumidor$ ./ejercicio03
[P0] Producing 0 ...
[P0] Producing 1 ...
[P1] Producing 0 ...
[P1] Producing 1 ...
[P2] Producing 0 ...
-----> [C0] consumed 0
-----> [C0] consumed 1
-----> [C1] consumed 0
-----> [C1] consumed 1
-----> [C2] consumed 0
[P2] Producing 1 ...
-----> [C2] consumed 1
[P1] Producing 2 ...
[P1] Producing 3 ...
-----> [C0] consumed 2
-----> [C2] consumed 3
[P2] Producing 2 ...
[P2] Producing 3 ...
-----> [C2] consumed 2
-----> [C1] consumed 3
[P0] Producing 2 ...
-----> [C1] consumed 2
[P0] Producing 3 ...
-----> [C0] consumed 3

```

- **Explicación del comportamiento**

En la ejecución del ejercicio 3 se observa el funcionamiento de tres productores y tres consumidores, identificados como P0, P1, P2 y C0, C1, C2. Los productores generan valores del 0 al 3 y los consumidores los van retirando del buffer. El orden de ejecución cambia entre los hilos, debido a que trabajan de manera concurrente. Sin embargo, los semáforos permiten controlar el acceso al buffer y evitan que dos hilos lo utilicen de forma incorrecta al mismo tiempo. De esta manera, los productores y consumidores pueden trabajar de forma sincronizada sin perder los datos almacenados.